

心光伴旅 — 灵感速记与桌宠陪伴应用

小组 63 | 蔡欣雨 屠礼洁 李继轩 | 程序设计实习大作业报告

一、程序功能介绍

1.1 项目概述

"心光伴旅" (Heartlight Companion) 是一款基于 Qt 6 的桌面应用程序，旨在为用户提供灵感速记与虚拟宠物陪伴相结合的日常使用体验。用户可以通过悬浮速记窗口快速记录想法和心情，同时桌面上会有一只可爱的像素宠物以动画形式实时响应用户的操作状态，提供温暖的情感陪伴。

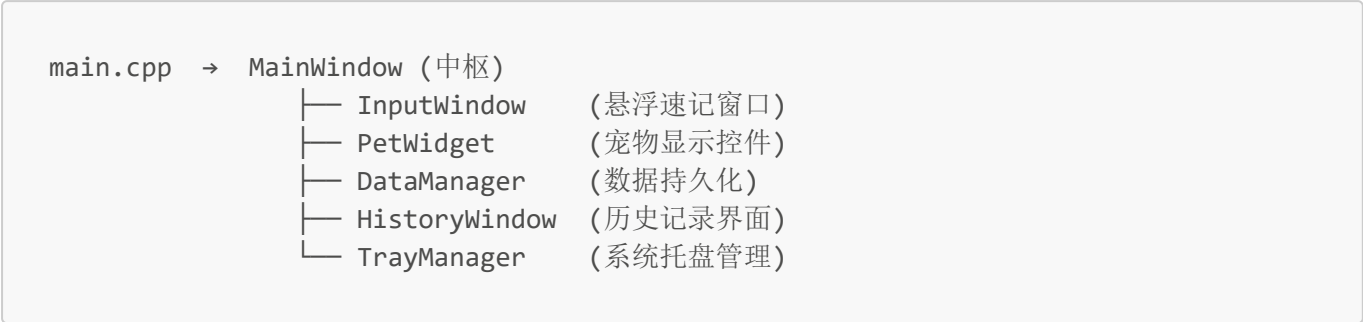
1.2 功能列表

序号	功能	说明
1	悬浮速记窗口	无边框、可拖拽的悬浮输入面板，Enter 键快捷保存，Shift+Enter 换行
2	心情选择	6 种心情下拉选择：Neutral / Happy / Calm / Excited / Unhappy / Focused
3	宠物 4 态动画	Idle（空闲）、Recording（记录中）、Success（保存成功）、Normal（点击互动），通过 QMovie 加载 GIF 动画
4	GIF 自适应尺寸	监听 QMovie::frameChanged 信号，窗口自动匹配 GIF 原始尺寸
5	闹钟提醒	输入时可勾选"Remind Me"，设置精确到分钟的时间，到点强制置顶弹窗
6	本地加密存储	数据以 XOR 流加密 + Base64 编码存入 .ett 文件，密钥随机生成
7	历史记录检索	支持关键字搜索、时间段筛选（全部/今天/本周/本月）、查看、编辑、删除、导出 TXT
8	密码保护	SHA-256 哈希存储密码，首次访问历史记录时强制设置
9	系统托盘	托盘菜单：快速打开速记、历史、备份、恢复、隐藏宠物、退出
10	托盘双击	双击托盘图标直接查看今日记录（通过密码验证）
11	每日治愈文案	桌宠空闲时随机展示一条原创中文鼓励语（每日首次，避免与昨日重复）
12	全局热键	Windows 用 RegisterHotKey (Ctrl+Alt+N/P)，macOS 用 Carbon 框架，支持唤起速记和切换宠物

二、项目各模块与类设计

2.1 整体架构

项目采用 **MainWindow 为中枢** 的架构设计，各功能模块通过 **Qt 信号-槽机制** 实现松耦合通信。整体结构如下：



MainWindow 负责创建所有子模块实例、连接信号槽、处理全局热键事件和闹钟轮询。各子模块之间不直接依赖，所有跨模块通信均通过 MainWindow 转发。

2.2 DataManager — 数据管理模块

文件: `DataManager.h` / `DataManager.cpp`

数据结构 — Record:

```
struct Record {
    QString id;           // UUID 唯一标识
    QString content;      // 速记内容
    QString mood;         // 心情标签
    QString type;         // 记录类型 ("Note")
    QDateTime timestamp;  // 创建时间
    bool hasAttachment;   // 是否有附件
    bool hasAlarm;        // 是否设置闹钟
    QDateTime alarmTime;  // 闹钟时间
};
```

核心设计:

- **加密存储:** 首次运行时调用 `ensureEncryptionKey()`，通过 `QRandomGenerator` 生成 32 字节随机密钥，Base64 编码后存入 `QSettings("security/encKey")`。数据序列化为 JSON 后经 XOR 流加密 + Base64 编码写入 `.ett` 文件。
- **CRUD 操作:** `addRecord()` 将新记录 prepend 到列表头部，`updateRecord()` 通过 id 匹配更新，`removeRecord()` 使用 `std::remove_if` 按 id 删除。
- **搜索过滤:** `search(query, timeFilter)` 同时支持关键字匹配 (content/mood) 和时间段筛选 (Today / This Week / This Month)。
- **备份恢复:** `backupToFile()` 直接复制数据文件，`restoreFromFile()` 替换后重新加载。

关键方法:

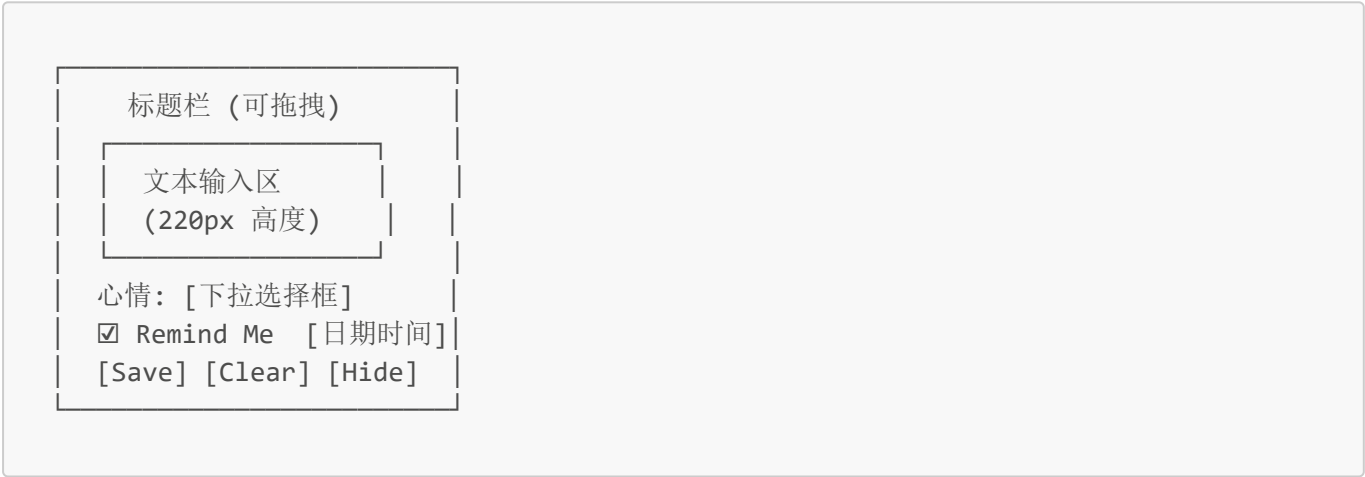
方法	功能
<code>ensureEncryptionKey()</code>	随机生成或读取持久化密钥

方法	功能
<code>encrypt()</code> / <code>decrypt()</code>	XOR 流加密 / 解密
<code>addRecord</code> / <code>updateRecord</code> / <code>removeRecord</code>	CRUD 操作
<code>search()</code>	关键字 + 时间段组合搜索
<code>exportToTxt()</code>	导出纯文本备份
<code>recordToJson()</code> / <code>jsonToRecord()</code>	结构体与 JSON 互转（含闹钟字段）

2.3 InputWindow — 速记输入模块

文件: `InputWindow.h` / `InputWindow.cpp`

UI 布局:



核心设计:

- **无边框可拖拽:** 使用 `Qt::FramelessWindowHint` 去除系统标题栏，通过 `mousePressEvent`/`mouseMoveEvent` 实现拖拽移动。标题栏安装 `eventFilter` 也可响应拖拽。
- **闹钟 UI:** 新增 `QCheckBox` ("Remind Me") 和 `QDateTimeEdit` (精确到分钟，默认禁用)。勾选复选框时启用日期选择器，保存时写入 `Record.hasAlarm` 和 `Record.alarmTime`。
- **Enter 快捷键:** 通过 `eventFilter` 拦截文本框的 `KeyPress` 事件，非 `Shift` 修饰时直接触发 `onSaveClicked()`。
- **样式表作用域:** `loadStyleSheet()` 中优先使用本地 `pet/` 目录下的 `style.qss` 文件，若不存在则使用 `qrc` 嵌入资源，通过 `this->setStyleSheet()` 仅影响自身。
- **文本样式持久化:** 文字颜色和字体保存到 `QSettings("ui/textColor", "ui/textFont")`，启动时自动恢复。

关键方法:

方法	功能
<code>createLayout()</code>	构建 UI 布局（含闹钟行）
<code>onSaveClicked()</code>	校验内容 → 构造 <code>Record</code> → 发射 <code>recordSaved</code> 信号 → 清空输入
<code>eventFilter()</code>	处理标题栏拖拽、文本框 Enter 保存、焦点占位符切换

方法	功能
<code>clearInputs()</code>	清空文本框 + 重置闹钟复选框和日期

2.4 PetWidget — 宠物展示模块

文件: `PetWidget.h` / `PetWidget.cpp`

核心设计:

- **4 态状态机:**
 - `setStateIdle()`: 随机加载 pet1~4.gif, 若无 GIF 则显示 "🐾 + 心情"
 - `setStateRecording()`: 优先加载 pet2.gif (打字动画), fallback "👉 + 心情"
 - `setStateSuccess()`: 优先加载 pet1.gif (庆祝动画), fallback "🎉 + 心情"
 - `setStateNormal()`: 停止动画, 显示 "🐱 + 心情"
- **GIF 自适应尺寸:** 在构造函数中监听 `QMovie::frameChanged`, 获取 GIF 原始尺寸后动态调整窗口大小。
窗口 = 上方 45px 气泡区 + GIF 实际高度。
- **每日治愈文案:** 构造函数内置 20 条原创中文鼓励语 (`kDailyQuotes`), `showDailyQuoteIfFirst()` 通过 `QSettings("dailyQuote/lastDate")` 判断当日是否已显示。随机选取一条 (避免与昨日重复), 用白色半透明气泡在窗口顶部居中展示 3 秒后自动消失。
- **气泡系统:** 点击宠物时在顶部居中展示随机鼓励语气泡, 1 秒后消失。
- **右键隐藏:** `mousePressEvent` 中右键直接 `hide()`。
- **左键拖拽:** `mouseMoveEvent` 中根据 `m_dragPosition` 计算新位置。
- **GIF 搜索路径:** `toLocalPetPath()` 在多个候选路径中依次查找 GIF 文件 (`applicationDirPath`、上级目录、当前工作目录等)。

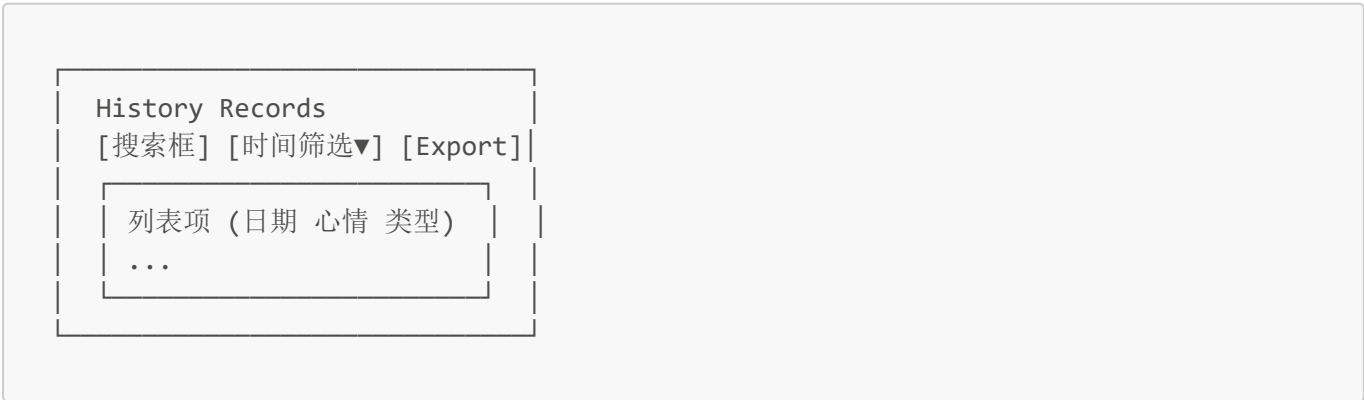
信号:

信号	触发时机
<code>petClicked()</code>	左键点击
<code>openInputRequested()</code>	左键点击 (同时发射, MainWindow 连接以打开速记窗口)

2.5 HistoryWindow — 历史记录模块

文件: `HistoryWindow.h` / `HistoryWindow.cpp`

UI 布局:



核心设计:

- **搜索与筛选联动:** `m_searchInput` 文本变化和 `m_timeFilter` 下拉变化均触发 `refreshRecords()`，调用 `DataManager::search()` 获取过滤后的记录列表。
- **记录详情弹窗:** 点击列表项弹出 `QMessageBox` 显示完整内容，提供"Edit"、"Delete"、"Close"三个按钮。Edit 使用 `QInputDialog::getMultiLineText` 编辑后调用 `updateRecord()`；Delete 二次确认后调用 `removeRecord()`。
- **导出 TXT:** `onExportClicked()` 调用 `QFileDialog::getSaveFileName` 选择路径，通过 `DataManager::exportToTxt()` 写入文件。
- **外部时间筛选:** 提供 `setTimeFilter(filter)` 公开方法，允许外部（如 `MainWindow` 的 `showTodayRecords()`）直接设置筛选下拉框的值。

2.6 TrayManager — 托盘管理模块

文件: `TrayManager.h` / `TrayManager.cpp`

核心设计:

- **自定义托盘图标:** `makeTrayIcon()` 使用 `QPainter` 在 32×32 `QPixmap` 上程序化绘制蓝紫渐变圆角背景 + 白色贝塞尔心形 + 金色星光，无需任何外部资源文件。
- **托盘菜单:** 构建包含"Open Note"、"Open History"、"Backup Data"、"Restore Data"、"Hide Pet"、"Quit"的上下文菜单，通过信号将操作路由到 `MainWindow`。
- **双击信号:** 连接 `QSystemTrayIcon::activated(DoubleClick)` 到 `todayRecordsRequested()` 信号，`MainWindow` 连接该信号以直接查看今日记录。
- **备份恢复:** `onBackup()` 和 `onRestore()` 使用 `QFileDialog` 获取路径，通过 `DataManager` 执行操作。

信号:

信号	触发时机
<code>showHistoryWindowRequested()</code>	托盘菜单点击"Open History"
<code>todayRecordsRequested()</code>	托盘图标双击
<code>backupRequested()</code>	托盘菜单点击"Backup Data"
<code>restoreRequested()</code>	托盘菜单点击"Restore Data"
<code>quitRequested()</code>	托盘菜单点击"Quit"

2.7 MainWindow — 中枢控制模块

文件: `mainwindow.h` / `mainwindow.cpp`

核心职责:

1. **创建并管理所有子模块:** 在构造函数中创建 `InputWindow`、`PetWidget`、`DataManager`、`HistoryWindow`、`TrayManager` 实例。
2. **信号-槽连接中心:** 连接 10+ 条信号-槽关系（模块间通信全部由此中转）。
3. **密码验证:** `showHistoryWindowProtected()` 实现完整的"设置密码 → 验证密码 → 打开历史"流程。密码以 SHA-256 哈希存入 `QSettings("security/passwordHash")`。

4. 闹钟轮询系统: 构造函数中创建 `m_alarmTimer` (1 秒间隔), 连接到 `checkAlarms()`。该方法遍历 `m_activeAlarms` 列表, 当前时间 $\geq$ 闹钟时间时触发置顶弹窗, 并联动宠物显示"Focused"心情。
5. 跨平台全局热键: Windows 平台使用 `RegisterHotKey` + `nativeEvent(WM_HOTKEY)` 实现系统级热键; macOS 平台在 `main.cpp` 中通过 Carbon `RegisterEventHotKey` 实现。析构函数中调用 `UnregisterHotKey` 释放资源。
6. 文字样式管理: 提供字体和颜色选择对话框, 设置应用到 `InputWindow` 和全局按钮样式。
7. 宠物可见性切换: `togglePetVisibility()` 切换显示/隐藏, 同步更新按钮文本。

关键信号-槽连接:

源模块	信号	目标	槽
InputWindow	<code>recordSaved</code>	MainWindow	<code>handleRecordSaved</code>
InputWindow	<code>windowShown</code>	PetWidget	<code>setStateRecording</code>
InputWindow	<code>windowHided</code>	PetWidget	<code>setStateIdle</code>
PetWidget	<code>petClicked</code>	MainWindow	<code>handlePetClicked</code>
PetWidget	<code>openInputRequested</code>	MainWindow	<code>showInputWindow</code>
DataManager	<code>recordsChanged</code>	MainWindow	<code>refreshHistory</code>
TrayManager	<code>showHistoryWindowRequested</code>	MainWindow	<code>showHistoryWindowProtected</code>
TrayManager	<code>todayRecordsRequested</code>	MainWindow	<code>showTodayRecords</code>

2.8 安全设计

层级	机制	实现
密码 存储	SHA-256 哈希	<code>QCryptographicHash::hash(password, Sha256).toHex()</code> 存入 <code>QSettings("security/passwordHash")</code>
加密 密钥	随机 32 字 节	首次运行 <code>QRandomGenerator</code> 生成, Base64 持久化到 <code>QSettings("security/encKey")</code>
数据 加密	XOR 流加 密	明文逐字节与密钥异或, 结果 Base64 编码写入 <code>.ett</code> 文件

2.9 跨平台全局热键方案

平台	实现方式	热键 1	热键 2
Windows	<code>RegisterHotKey</code> + <code>nativeEvent(WM_HOTKEY)</code>	<code>Ctrl+Alt+N</code> → 速记窗口	<code>Ctrl+Alt+P</code> → 切换宠物
macOS	Carbon <code>RegisterEventHotKey</code>	<code>Ctrl+Option+N</code> → 速记窗口	<code>Ctrl+Option+P</code> → 切换宠物

macOS 热键在 `a.exec()` 返回后调用 `UnregisterEventHotKey` 释放资源, 防止泄漏。

三、小组成员分工

成员	负责模块	具体工作内容
蔡欣雨	数据层	DataManager 类设计（Record 结构体、XOR 加密、随机密钥生成、JSON 序列化、CRUD 操作、搜索过滤、备份恢复）；密码 SHA-256 验证逻辑；安全方案设计与实现
屠礼洁	UI 交互层	InputWindow 界面与交互（悬浮窗口、心情选择、闹钟 UI、Enter 快捷键、事件过滤器）；HistoryWindow 界面（搜索筛选、列表展示、查看编辑删除、导出 TXT、setTimeFilter 外部接口）；TrayManager 托盘系统（QPainter 自定义图标、菜单构建、双击信号、备份恢复操作）
李继轩	表现层与控制层	PetWidget 宠物展示（4 状态状态机、QMovie GIF 加载、GIF 自适应尺寸、每日治愈文案系统、气泡交互、拖拽移动）；MainWindow 主控逻辑（模块创建、全部信号-槽连接、闹钟轮询系统、文字样式管理、宠物可见性控制）；main.cpp 入口与跨平台热键（Windows RegisterHotKey、macOS Carbon、热键资源释放）

注：三人分工按数据层 / UI 交互层 / 表现与控制层划分，各自职责清晰，通过 Qt 信号-槽机制实现模块间解耦协作。

四、项目总结与反思

4.1 技术收获

- 1. **Qt 信号-槽机制**: 深刻理解了信号-槽在复杂桌面应用中的解耦价值。6 个独立模块通过 MainWindow 中的信号-槽连接实现协作，各模块无需知道彼此的存在。
- 2. **跨平台开发**: 通过条件编译（`#ifdef Q_OS_WIN / #ifdef Q_OS_MAC`）分别实现了 Windows 的 RegisterHotKey 和 macOS 的 Carbon EventHotKey，体会到不同平台 API 的差异和适配方法。
- 3. **数据安全实践**: 从硬编码密钥演进到随机密钥 + QSettings 持久化，从明文密码到 SHA-256 哈希，理解了安全分层设计的重要性。
- 4. **QPainter 程序化绘图**: 通过贝塞尔曲线绘制心形图案，无需依赖外部图片资源即可生成精美的托盘图标。

4.2 难点与解决方案

难点	解决方案
Windows 全局热键无效（QShortcut 仅应用级）	改用 <code>RegisterHotKey + nativeEvent(WM_HOTKEY)</code> 实现系统级热键
macOS 热键资源泄漏	将 <code>EventHotKeyRef</code> 提升为文件作用域静态变量，在 <code>a.exec()</code> 后 <code>UnregisterEventHotKey</code>
输入窗口样式影响全局	<code>loadStyleSheet()</code> 从 <code>qApp->setStyleSheet()</code> 改为 <code>this->setStyleSheet()</code> ，限定作用域
宠物窗口尺寸与 GIF 不匹配	监听 <code>QMovie::frameChanged</code> 信号，动态读取 GIF 原始尺寸并调整窗口

难点	解决方案
旧数据文件与新加密密钥不兼容	提供 TXT 导出备份方案
闹钟提醒需要应用关闭后仍有效	当前实现依赖应用运行中的 QTimer 轮询，这是本版本的局限（见 4.3）

4.3 不足与改进方向

1. **闹钟持久化:** 当前闹钟仅在程序运行期间生效，关闭后失效。未来可考虑使用操作系统原生定时任务（如 Windows Task Scheduler、macOS launchd）或 Qt 的 `QProcess` 调用系统命令实现持久化提醒。
2. **云同步:** 当前数据仅存储在本地 `.ett` 文件。可扩展支持云端备份（如通过 REST API 上传加密数据到服务器）。
3. **宠物行为丰富度:** 当前宠物仅支持 4 种状态和被动响应。可增加自主行为（定时随机走动、根据系统时间切换昼夜动画）、喂食/抚摸等交互。
4. **UI 动画过渡:** 宠物状态切换时缺少平滑的过渡动画，可通过 `QPropertyAnimation` 实现淡入淡出效果。
5. **多语言国际化:** 当前仅支持中文界面，可引入 Qt Linguist 实现多语言支持。

4.4 协作体会

本次大作业采用 Git + GitHub 进行版本控制协作。demo1 版本在 final 版本基础上进行了大量安全加固、功能增强和代码优化，最终融合了两个版本的优势，形成了一个功能完整、安全可靠、用户体验良好的桌面应用。

附录：项目文件清单

文件	行数	说明
<code>main.cpp</code>	76	程序入口、macOS Carbon 热键
<code>mainwindow.h</code>	64	主窗口声明（含 <code>ActiveAlarm</code> 结构体）
<code>mainwindow.cpp</code>	277	主窗口实现（信号连接、密码、热键、闹钟）
<code>InputWindow.h</code>	68	速记窗口声明（含闹钟控件）
<code>InputWindow.cpp</code>	264	速记窗口实现（UI、事件处理、保存逻辑）
<code>PetWidget.h</code>	48	宠物控件声明
<code>PetWidget.cpp</code>	262	宠物控件实现（动画、文案、GIF 自适应）
<code>DataManager.h</code>	52	数据管理声明（ <code>Record</code> 结构体）
<code>DataManager.cpp</code>	250	数据管理实现（加密、CRUD、搜索）
<code>HistoryWindow.h</code>	44	历史窗口声明
<code>HistoryWindow.cpp</code>	167	历史窗口实现（搜索筛选编辑导出）
<code>TrayManager.h</code>	52	托盘管理声明
<code>TrayManager.cpp</code>	94	托盘管理实现（图标绘制、菜单、双击）
<code>style.qss</code>	—	全局样式表

文件	行数	说明
resources.qrc	—	Qt 资源文件（样式 + GIF）
demo1.pro	37	Qt 项目配置文件
README.md	—	项目说明文档